the tab pages according to our choice? This can certainly be done by using the following set of commands:

```
:tabmove
:tabmove 0
:tabmove [N]
:tabm
:tabm 0
:tabm [N]
```

The *:tabmov* command moves the current tab page to the last position. To move the current tab page to the first position, execute the following command:

```
:tabmove 0
```

Similarly, the *:tabmove [N]* command moves the current tab page to the N + 1th position. Please note that the *:tabmove* command counts tab pages from zero. Hence, *:tabmove 4* moves the current tab page to the fifth position. *:tabm* is just the abbreviated form of the *:tabmove* command.

The *:tabdo* or *:tabd* commands are extremely useful when we want to execute certain commands in each tab page. Here is the syntax of the command:

```
:tabdo <command>
:tabd <command>
```

Here, the angular bracket implies the command is mandatory. Listed below is the workflow of the *:tabdo* command.

Step 1: Go to the first tab page.
Step 2: Execute the specified command in the current tab page.
Step 3: If it is a last tab page then stop the execution of the command.
Step 4: Otherwise jump to the next tab page and go to Step 2.

Let us suppose that we have opened multiple tab pages and want to replace the word 'Tom' with 'Jerry' on each tab page; then the following command will do the needful:

```
:tabdo %s/\<Tom\>/Jerry
```

Please note that one of the limitations of this command is that it can only execute the same command in each tab page.

Vim already supports the command that works nicely with split windows. Vim provides the *:tab* command line modifier to use a new tab page instead of a new window for commands that would normally split a window. For instance, the *:tab ball* command shows each buffer in a separate tab page. The simple example given below illustrates the same.

```
[bash]$ vim file1.txt file2.txt file3.txt
```

In the above command, Vim opens three files in separate buffers but only shows a single buffer/ window at a time. We can use the *:bnext* command to switch to the next buffer. Now, to open these buffers into separate tab pages, execute the *:tab ball* command from the current Vim session.

Additionally, we can view Vim's help contents in a new tab page. By default, the *:help* command splits the window horizontally and displays help contents in the split window. To view help contents in a separate tab page, execute the following command from the Vim session, as shown below:

```
:tab help
```

Similarly, the *:tab split [filename]* command opens a file in a new tab page rather than splitting he window. If the *filename* is omitted with this command, it copies the current window to a new tab page. Otherwise, it opens the specified file in a new tab page. For instance, the command below opens the *file1.txt* file in a new tab page.

```
[bash]$ vim file1.txt # Now execute the command below from Vim
:tab split
```

Wouldn't it be great if we could use both tab pages and split window features together? Vim provides that flexibility. Let us suppose you have opened several files in a split window and you want to move a certain file to a new tab page. Just press *Ctrl+W T*. Similarly, to close the current tab page, use the *Ctrl+W c* key combination.

We can also use the *goto* file feature with the tab pages. By using this, we can put the cursor on the file name and open the file into a new tab page by pressing the *Ctrl+W gf* key combination. The example below illustrates this feature:

```
[bash]$ echo "This is file#1" > file1.txt

[bash]$ echo "file1.txt" > file2.txt

[bash]$ vim file2.txt # Now execute the command below from Vim
Ctrl+W gf or
Ctrl+W gF
```

This will open *file1.txt* in a new tab page. Please note that you must set the appropriate 'path' option to use this feature.

Myriad tasks can be done with Vim. Only a few keystrokes are required to solve complex tasks. These simple yet powerful features really make Vim more interesting. To gain insights into tab pages, go through Vim's help documentation. You can access the document by executing the *:help tab-page-intro* command from the Vim session. END

By: Narendra Kangralkar

The author is a FOSS enthusiast and loves exploring anything related to open source. He can be reached at narendrakangralkar@gmail.com